

Project 08 — LSTM Time Series Forecasting

Level: Intermediate | **Dataset:** Stock Price (Yahoo Finance / Kaggle) | **Framework:** PyTorch

Objective

Build an LSTM for multi-step stock price forecasting.

Cover: sliding window creation, stateful vs stateless LSTM, multi-step output, sequence-to-sequence approach, LR warm-up.

Project Structure

```
08_lstm_timeseries/
├── notebooks/
│   ├── 01_eda_timeseries.ipynb
│   ├── 02_windowing_preprocessing.ipynb
│   └── 03_train_evaluate.ipynb
├── data/raw/AAPL.csv
├── data/processed/
├── models/model.pkl
├── charts/
├── path_utils.py
├── dashboard_core.py
└── app.py
```

Dataset: Apple (AAPL) stock — download via `yfinance` or use Kaggle stock dataset. Use columns: Open, High, Low, Close, Volume. Target: predict next 5 Close values.

Notebook 01 — EDA (`01_eda_timeseries.ipynb`)

STOP 1 — Load & Plot

- Load AAPL.csv (or use `yfinance.download('AAPL', start='2015-01-01', end='2024-01-01')`)
- Plot Close price over time
- Plot Volume over time
- Print basic stats: mean, std, min, max of Close
- **Agent stops here. Explain:**
 - What time series data is: ordered observations over time
 - Why time series is different from tabular: temporal dependency between rows
 - What stationarity means: does the mean/variance change over time?
 - Why raw stock prices are non-stationary (trend upward)
- Wait for user confirmation before continuing

STOP 2 — Stationarity & Returns

- Compute daily returns: `df['Return'] = df['Close'].pct_change()`
- Plot returns over time
- Run ADF (Augmented Dickey-Fuller) test: `from statsmodels.tsa.stattools import adfuller`
- **Agent stops here. Explain:**
 - Why returns are more stationary than raw prices
 - What ADF test tells us: $p\text{-value} < 0.05 = \text{stationary}$
 - Why LSTM can sometimes learn non-stationary series but it's harder
 - The difference between forecasting price and forecasting return
- Wait for confirmation

STOP 3 — Feature Engineering for Time Series

Add features:

- `df['MA7'] = df['Close'].rolling(7).mean()` — 7-day moving average
 - `df['MA30'] = df['Close'].rolling(30).mean()`
 - `df['Volatility'] = df['Close'].rolling(7).std()`
 - Drop NaN rows from rolling windows
 - **Agent stops here. Explain:**
 - What moving averages capture: trend signal
 - What rolling std captures: recent volatility
 - Why we add these features even though LSTM can theoretically learn them
 - The data leakage risk: rolling window must only use past data (already the case here)
 - Wait for confirmation
-

Notebook 02 — Windowing & Preprocessing (`02_windowing_preprocessing.ipynb`)

STOP 4 — Temporal Train/Test Split

- Use time-based split: first 80% as train, last 20% as test
- **NEVER use random split for time series**
- **Agent stops here. Explain:**
 - Why random split causes data leakage in time series
 - What "look-ahead bias" is: using future data to predict past
 - How to properly create a val set: take last 10% of train (not random)
 - Why the test set must always be the most recent data
- Wait for confirmation

STOP 5 — MinMax Scaling (Temporal-Safe)

- Fit `MinMaxScaler` on train set only
- Transform train, val, test
- Scale all features including target (Close)
- Save scaler
- **Agent stops here. Explain:**
 - Why MinMax (not StandardScaler) is often preferred for time series: bounds to [0,1]
 - Why we MUST fit scaler only on train (fitting on test = leakage)
 - Why we scale the target for regression here (Close price range is large)
 - How to `inverse_transform` predictions back to original price scale
- Wait for confirmation

STOP 6 — Sliding Window Dataset

Write `create_sequences(data, seq_len=60, pred_len=5)`:

- For each position `i`: `X = data[i:i+seq_len]`, `y = data[i+seq_len:i+seq_len+pred_len, close_idx]`
- Return `X: [N, 60, num_features]`, `y: [N, 5]`
- **Agent stops here. Explain:**
 - What a sliding window is: a fixed-length lookback window that slides forward
 - What `seq_len=60` means: use 60 days of history to predict next 5 days
 - What `pred_len=5` means: multi-step forecasting (Monday to Friday next week)
 - How many samples we get: `len(data) - seq_len - pred_len`
 - Why this creates temporal overlap between consecutive windows (is that OK?)
- Wait for confirmation

Notebook 03 — Train & Evaluate (`03_train_evaluate.ipynb`)

STOP 7 — Multi-step LSTM Architecture

```
Input: [B, 60, num_features]
nn.LSTM(input_size=num_features, hidden_size=128, num_layers=2,
        batch_first=True, dropout=0.2)
→ Take output[:, -1, :] # last timestep hidden state [B, 128]
nn.Dropout(0.3)
nn.Linear(128, 64) → ReLU
nn.Linear(64, 5) # predict 5 timesteps ahead
```

- **Agent stops here. Explain:**
 - Why `output[:, -1, :]` not `h_n[-1]`: for multi-layer LSTM they are equivalent, but output is cleaner
 - What the output `Linear(64, 5)` represents: direct multi-step regression
 - Two strategies for multi-step:
 1. Direct (this approach): one shot, predict all 5 at once
 2. Recursive: predict 1, use prediction as input, repeat 5 times
 - Why direct is better in practice (errors don't compound)
- Wait for confirmation

STOP 8 — Sequence-to-Sequence Alternative

Implement alternative `Seq2SeqLSTM`:

- Encoder LSTM reads the 60-timestep input
- Decoder LSTM generates 5 output steps one at a time
- Teacher forcing: during training, feed true previous value to decoder
- **Agent stops here. Explain:**
 - What teacher forcing is: use ground truth as decoder input during training
 - Why teacher forcing speeds training (decoder doesn't propagate its own errors)
 - The exposure bias problem: at inference time, no ground truth → use predictions
 - When Seq2Seq is worth the complexity over direct: long prediction horizons (>20 steps)
- Wait for confirmation

STOP 9 — Learning Rate Warmup

Implement LR warmup:

python

```
def lr_lambda(step):
    warmup_steps = 100
    if step < warmup_steps:
        return step / warmup_steps
    return 0.5 * (1 + cos(pi * (step - warmup_steps) / (total_steps - warmup_steps)))
scheduler = LambdaLR(optimizer, lr_lambda)
```

- Call `scheduler.step()` after each batch (not epoch)
- **Agent stops here. Explain:**
 - What LR warmup does: start with tiny LR, ramp up, then decay
 - Why warmup helps: early training is unstable — small LR prevents large parameter jumps
 - Why cosine decay after warmup: smooth convergence
 - When warmup is essential: transformers, large models, large LR — also helps LSTMs
- Wait for confirmation

STOP 10 — Training Loop

- 50 epochs, batch_size=32 (sequences are larger)
- MSE loss on all 5 predicted steps
- Apply gradient clipping (max_norm=1.0)
- Plot: loss curve, LR schedule curve
- **Agent stops here. Explain:**
 - Why smaller batch for sequences (higher memory per sample)
 - How MSE is computed over 5 predicted values: mean across pred_len
 - Why gradient clipping is critical again here (LSTM + long sequences = exploding grad risk)
- Wait for confirmation

STOP 11 — Time Series Metrics

Compute on test set:

- MSE, RMSE, MAE (in scaled units)
- Inverse transform predictions → original price scale
- RMSE, MAE in dollars
- MAPE: Mean Absolute Percentage Error
- Plot: actual vs predicted prices for entire test period
- Plot: zoom-in on last 60 days
- **Agent stops here. Explain:**
 - Why MAPE can be misleading (division by actual → large when price is near 0)
 - What RMSE in dollars means practically (average prediction error in \$)
 - How to interpret "my model predicts within \$X of actual price"
 - Directional accuracy: does the model predict the correct direction (up/down)?
- Wait for confirmation

STOP 12 — Walk-Forward Validation

Implement walk-forward evaluation:

- Split test into weekly chunks
- For each chunk: retrain on all data up to that point, predict that week
- Compare RMSE: simple walk-forward vs fixed-train-window
- **Agent stops here. Explain:**
 - What walk-forward validation is: the correct way to backtest time series models
 - Why it's computationally expensive vs simple test split
 - When it's required: financial models, production forecasting systems
 - The concept of model drift: model trained in 2015 on 2023 market data
- Wait for confirmation

STOP 13 — Save & Forecast Function

- Save `model.state_dict()`, scaler
- Write `forecast_next5(recent_60_days_df)` → 5 future price predictions
- **Agent stops here. Explain:**
 - How to construct the input: take last 60 rows → scale → reshape `[1, 60, features]` → model → inverse scale → list of 5 prices
 - Why the most recent 60 days must be scaled using the SAME fitted scaler
- Wait for confirmation

`dashboard_core.py`

Functions:

- `load_model_and_scaler()` → model, scaler, feature_cols
 - `forecast_next5(df_recent)` → list of 5 price predictions
 - `get_training_curves()` → dict
 - `get_test_predictions()` → (dates, actual, predicted) arrays
 - `get_metrics()` → RMSE, MAE, MAPE dict
-

`app.py` — **Streamlit (~80 lines)**

Sections:

1. Load latest AAPL data automatically (or use saved test set)
 2. Show current price + forecast next 5 days as a chart
 3. Tab 1: Training loss curve
 4. Tab 2: Full test period actual vs predicted
 5. Tab 3: Metrics (RMSE, MAE, MAPE in dollars)
-

Key Concepts Covered

- Temporal train/test split (no random split)
- Sliding window sequence creation
- Multi-step direct forecasting vs Seq2Seq
- Teacher forcing and exposure bias
- LR warmup with cosine decay (LambdaLR)
- Walk-forward validation
- MAPE and directional accuracy
- Inverse transform for interpretable predictions